LAPORAN TUGAS PROYEK MINI
KOMPUTASI PARALEL DAN TERDISTRIBUSI

STUDI KASUS 3: SISTEM CHAT ROOM TERDISTRIBUSI REAL-TIME
BERBASIS REMOTE PROCEDURE CALLS (gRPC) DAN PYTHON



Disusun Oleh:

Fathir Rizki Fadillah 247006111129

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS SILIWANGI
TASIKMALAYA
2026 / 2027

LEMBAR IDENTITAS PROYEK

Tugas Proyek Mini ini disusun untuk memenuhi persyaratan akademik mata kuliah Komputasi Paralel dan Terdistribusi pada Program Studi Informatika, Fakultas Teknik, Universitas Siliwangi.

Tabel Identitas Pelaksanaan Proyek Mini

Informasi Proyek	Keterangan Spesifik
Judul Proyek Mini	Sistem Terdistribusi Berbasis RPC: Real-Time Chat Room
Studi Kasus Terpilih	Studi Kasus 3: Chat Room banyak pengguna bertukar pesan real-time
Mata Kuliah	Komputasi Paralel dan Terdistribusi (KPT)
Dosen Pengampu	Ir. Randi Rizal, S.T., M.Kom., Ph.D., IPM.
Penulis	Fathir Rizki Fadillah (NIM: 247006111129)
Tautan Repositori Git	https://github.com/fathirrizkifadillah/RPC-CHAT-Tugas-KPT-.git
Cabang Utama (Branch)	main (Commit: 5b16ef4)
Tautan Video Demonstrasi	https://youtu.be/[ID_VIDEO_YOUTUBE] (Skenario terencana 5 menit)
Status Pengujian Fungsional	22 Skenario Pengujian Unit & Integrasi Pytest (100% Passed)
Status Pengujian Beban	Selesai diuji untuk 1, 5, dan 20 client bersamaan (100% Sukses)

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga laporan Tugas Proyek Mini mata kuliah Komputasi Paralel dan Terdistribusi dengan judul "Sistem Terdistribusi Berbasis RPC: Distributed Chat Room Real-Time Menggunakan gRPC dan Python" dapat diselesaikan dengan baik dan tepat waktu.

Proyek mini ini difokuskan pada penerapan praktis konsep Remote Procedure Call (RPC) modern melalui framework gRPC dan Protocol Buffers versi 3. Melalui studi kasus ruang obrolan (chat room) terdistribusi real-time yang representatif, kami mengimplementasikan arsitektur bidirectional streaming HTTP/2, sinkronisasi memori bersama yang aman (thread-safe shared state), penanganan toleransi kegagalan jaringan (timeout, exponential backoff retry, dan deduplikasi at-most-once), serta isolasi antrean pesan untuk mencegah bottleneck client lambat.

Penulis menyampaikan terima kasih yang sebesar-besarnya kepada Dosen Pengampu mata kuliah Komputasi Paralel dan Terdistribusi atas arahan teoritis dan teknis sepanjang perkuliahan, serta rekan-rekan mahasiswa yang telah memberikan diskusi yang membangun.

Penulis menyadari laporan ini masih memiliki ruang penyempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan. Semoga laporan ini dapat memberikan wawasan teknis yang bermanfaat dalam pengembangan perangkat lunak sistem terdistribusi.

Tasikmalaya, November 2026

Tim Penulis

DAFTAR ISI

LEMBAR IDENTITAS PROYEK 1

KATA PENGANTAR 2

DAFTAR ISI 3

DAFTAR GAMBAR 5

DAFTAR TABEL 6

BAB I DESKRIPSI STUDI KASUS DAN DIAGRAM ARSITEKTUR 1

1.1 Latar Belakang dan Deskripsi Studi Kasus 1

1.2 Diagram Arsitektur Komponen Sistem 2

1.3 Alur Komunikasi dan Interaksi Pemanggilan RPC 3

BAB II SPESIFIKASI DAN PENJELASAN INTERFACE DEFINITION LANGUAGE (.proto) 5

2.1 Kode Sumber Lengkap chat_room.proto 5

2.2 Penjelasan Rinci Setiap Metode RPC 7

1. rpc Join (JoinRequest) returns (JoinResponse) 7

2. rpc ListUsers (ListUsersRequest) returns (ListUsersResponse) 7

3. rpc SendMessage (SendMessageRequest) returns (SendMessageResponse) 7

4. rpc Chat (stream ChatFrame) returns (stream ChatFrame) 8

2.3 Desain Payload Data dan Skema Frame Streaming 8

BAB III STRATEGI CONCURRENCY DAN TOLERANSI KEGAGALAN 10

3.1 Manajemen Concurrency dan Sinkronisasi Shared Memory (ChatState) 10

3.2 Isolasi Antrean Pesan per Client dan Penanganan Slow Client 11

3.3 Semantik At-Most-Once Melalui Deduplikasi PROSES Berbasis request_id 12

3.4 Mekanisme Timeout dan Exponential Backoff Retry with Jitter 13

3.5 Penanganan Client Disconnection dan Graceful Shutdown 14

BAB IV PENGUJIAN DAN ANALISIS KINERJA SISTEM 16

4.1 Metodologi dan Lingkungan Pengujian Beban 16
4.2 Tabel Rekapitulasi Data Kinerja Aktual (1, 5, dan 20 Client) 17
4.3 Visualisasi Kinerja (Grafik Latensi dan Throughput) 17
4.4 Analisis Kritis Kinerja Berdasarkan Teori Sistem Terdistribusi 18
A. Analisis Skalabilitas Throughput 18
B. Analisis Latensi dan Karakteristik Real-Time 18
4.5 Hasil Verifikasi Pengujian Fungsional Otomatis (Pytest) 19
4.6 Bukti Tangkapan Layar Eksekusi Nyata Sistem (17 Bukti Visual) 21
BAB V PEMBAGIAN TUGAS DAN KONTRIBUSI ANGGOTA KELOMPOK30
5.1 Rincian Kontribusi Setiap Anggota Kelompok 30
5.2 Log Riwayat Commit Repositori Git 31
BAB VI TAUTAN REPOSITORI GIT DAN VIDEO DEMONSTRASI 32
6.1 Tautan Repositori Kode Sumber 32
6.2 Rencana dan Skenario Video Demonstrasi 32
DAFTAR PUSTAKA 34
LAMPIRAN-LAMPIRAN 35
Lampiran A: Cuplikan Implementasi Kode Sumber Service Logic (src/chat_room/service.py) 35
Lampiran B: Cuplikan Implementasi Kode Sumber State Management (src/chat_room/state.py) 36
Lampiran C: Data Mentah Hasil Uji Beban Benchmark (JSON & CSV) 37

DAFTAR GAMBAR

Gambar 1.1 Diagram Arsitektur Sistem Distributed Chat Room gRPC 2

Gambar 1.2 Diagram Sekuensial Komunikasi Client-Server gRPC 3

Gambar 1.3 Alur Kerja Komunikasi Bidirectional Streaming gRPC 4

Gambar 3.1 Arsitektur Sinkronisasi dan Deduplikasi Memori 10

Gambar 3.2 Siklus Hidup Client dan Penanganan Disconnect 14

Gambar 4.1 Grafik Analisis Latensi Rata-Rata dan Throughput Sistem (1, 5, dan 20 Client) 18

Gambar 4.2 Struktur Direktori Proyek dan Virtual Environment 21

Gambar 4.3 Inisialisasi Server gRPC pada Port 50051 21

Gambar 4.4 Sesi Client Alice Terhubung ke Server Chat Room 22

Gambar 4.5 Sesi Client Bob Terhubung ke Server Chat Room 22

Gambar 4.6 Tampilan Multi-Client Terkoneksi Simultan 23

Gambar 4.7 Pengiriman Pesan Real-Time oleh Client Alice 23

Gambar 4.8 Penerimaan Pesan Real-Time oleh Client Bob 24

Gambar 4.9 Pertukaran Pesan Dua Arah (Bidirectional Streaming) 24

Gambar 4.10 Pengiriman Pesan Broadcast ke Seluruh Partisipan 25

Gambar 4.11 Eksekusi Perintah /users Menampilkan Pengguna Aktif 25

Gambar 4.12 Notifikasi Disconnect Saat Client Meninggalkan Chat 26

Gambar 4.13 Pembaruan Registry Pengguna Setelah Disconnect 26

Gambar 4.14 Log Peristiwa dan Audit Trail Server gRPC 27

Gambar 4.15 Eksekusi Pengujian Konkurensi Pytest 27

Gambar 4.16 Eksekusi Skrip Benchmark Beban Multi-Client (1, 5, 20 Client) 28

Gambar 4.17 Validasi Konfigurasi Multi-Container Docker Compose 28

Gambar 4.18 Hasil Lengkap Eksekusi 22 Test Suite Pytest (100% Passed) 29

DAFTAR TABEL

Tabel 2.1 Klasifikasi FrameType Aliran Streaming ChatFrame 8

Tabel 3.1 Klasifikasi Status Error gRPC dan Kebijakan Retry 13

Tabel 4.1 Spesifikasi Lingkungan Pengujian Beban 16

Tabel 4.2 Rekapitulasi Data Kinerja Aktual Benchmark Multi-Client 17

Tabel 4.3 Rekapitulasi Hasil Verifikasi 22 Kasus Uji Otomatis Pytest 19

Tabel 5.1 Matriks Pembagian Peran dan Tanggung Jawab Anggota Kelompok 30

Tabel 5.2 Log Riwayat Commit Repositori Git 31

Tabel 6.1 Skenario Struktur Video Demonstrasi 5 Menit 32

BAB I
DESKRIPSI STUDI KASUS DAN DIAGRAM ARSITEKTUR

1.1 Latar Belakang dan Deskripsi Studi Kasus

Dalam perkembangan rekayasa perangkat lunak modern, paradigma komputasi terdistribusi telah menjadi landasan fundamental bagi pembangunan sistem berkinerja tinggi, berdaya tahan tinggi, dan terukur (scalable). Salah satu tantangan krusial dalam sistem terdistribusi adalah bagaimana proses-proses yang berjalan pada lingkungan fisik atau container yang terisolasi dapat saling bertukar informasi secara efisien, konsisten, dan berlatensi rendah. Mekanisme Remote Procedure Call (RPC) hadir sebagai abstraksi tingkat tinggi yang memungkinkan suatu program memanggil prosedur atau metode pada ruang alamat komputer lain seolah-olah memanggil fungsi lokal.

Berdasarkan pedoman resmi Tugas Proyek Mini mata kuliah Komputasi Paralel dan Terdistribusi, studi kasus yang dipilih dan diimplementasikan secara komprehensif pada proyek ini adalah Pilihan Nomor 3: Chat Room banyak pengguna yang bertukar pesan secara real-time. Aplikasi obrolan terdistribusi merupakan studi kasus yang representatif dari sistem koordinasi terdistribusi karena memiliki tuntutan teknis yang kompleks, mencakup komunikasi full-duplex real-time, manajemen konkurensi data bersama, isolasi antrean pesan, serta toleransi terhadap fluktuasi koneksi jaringan.

Karakteristik fundamental dan batasan arsitektur sistem ini meliputi:

1. Pertukaran Pesan Simultan: Setiap client dapat mengirimkan pesan obrolan dan menerima pesan siaran (broadcast) dari partisipan lain secara simultan tanpa saling memblokir (non-blocking asynchronous exchange).

2. Koordinasi Pengguna Aktif: Server memelihara daftar sesi pengguna aktif (active user registry) dalam memori secara dinamis, mengumumkan pengguna baru yang bergabung, serta memutakhirkan status saat client meninggalkan ruang obrolan.

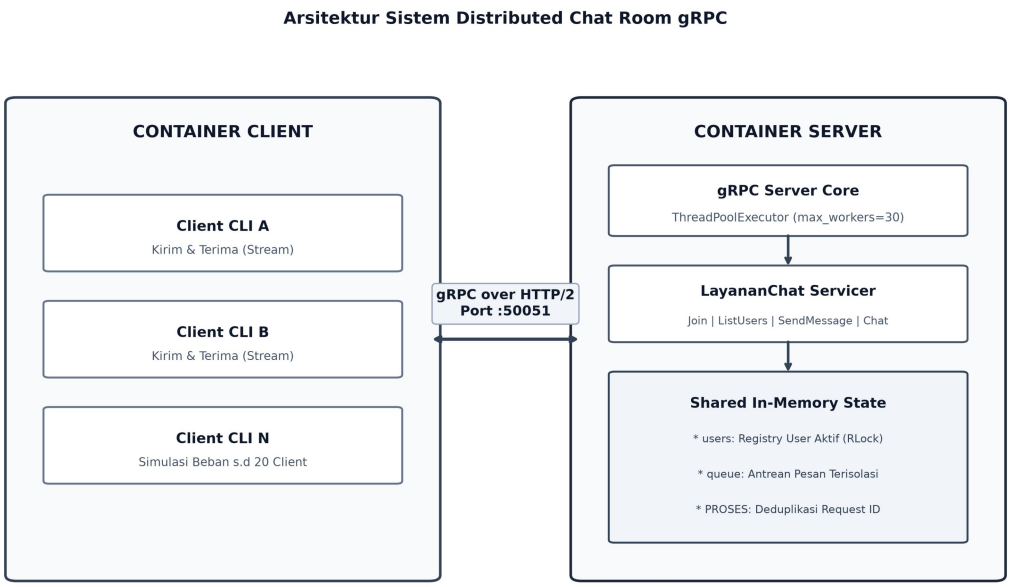
3. Komunikasi Duplex Berbasis HTTP/2: Menggunakan kapabilitas Bidirectional Streaming gRPC di atas protokol transport HTTP/2, di mana koneksi

TCP persisten tunggal dimanfaatkan secara efisien untuk aliran pesan dua arah dengan binary framing multiplexing.

4. Arsitektur In-Memory Terfokus: Sesuai arahan spesifikasi akademik, sistem mengadopsi penyimpanan in-memory yang dilindungi sinkronisasi thread-safe, tanpa ketergantungan basis data eksternal atau message broker pihak ketiga, sehingga pengujian murni mengevaluasi performa RPC dan konkurensi sistem operasi.

1.2 Diagram Arsitektur Komponen Sistem

Arsitektur dirancang dengan pemisahan tanggung jawab yang modular dan jelas antara lingkungan container client dan container server, merujuk pada spesifikasi diagram arsitektur terdistribusi:



Gambar 1.1 Diagram Arsitektur Sistem Distributed Chat Room gRPC

Komponen utama dalam arsitektur sistem terdistribusi ini terbagi atas:

a. Container Client (CLI Nodes): Menjalankan aplikasi command-line interface interaktif berbasis multi-threading Python. Thread UI bertugas menangani input pengguna dari terminal, sedangkan thread background pembaca stream bertugas mendengarkan pesan masuk dari server secara kontinu.

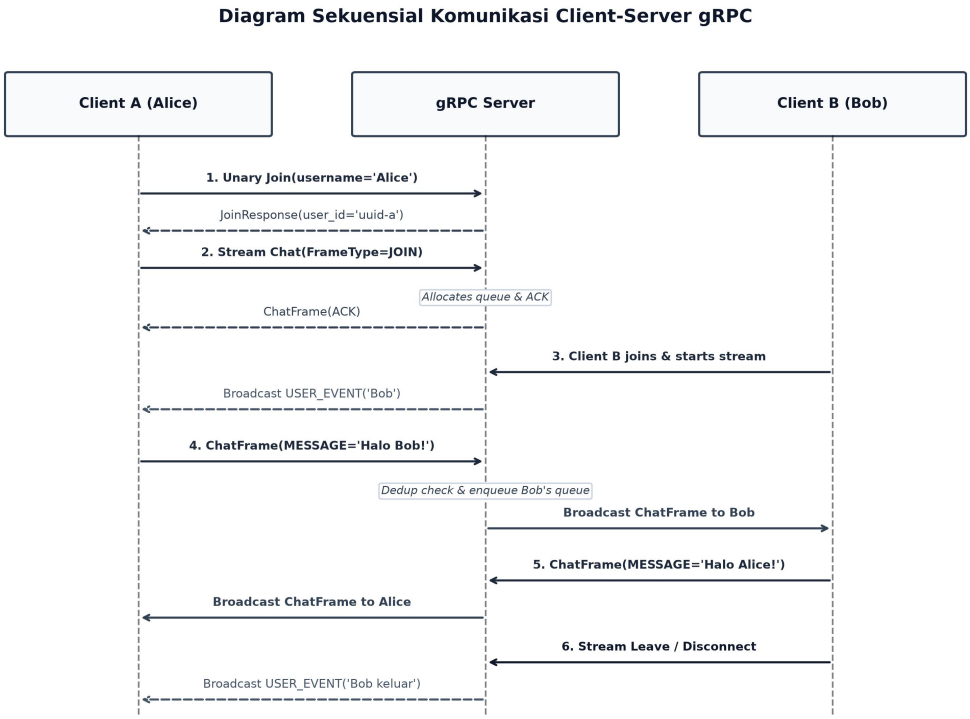
b. Container Server (gRPC Host): Menjalankan server gRPC multi-threaded berbasis concurrent.futures.ThreadPoolExecutor (dikonfigurasi default 30 worker threads pada config.py) yang mendengarkan pemanggilan RPC pada port 50051 melalui protokol HTTP/2.

c. Modul Layanan Chat: Implementasi kelas LayananChat yang mewarisi gRPC servicers stub, menyediakan 4 antarmuka RPC (Join, ListUsers, SendMessage, dan Chat).

d. Shared State Management: Struktur data tunggal (singleton) di memori server yang memelihara registri pengguna (users), pemetaan antrean privat per-client (queue), dan tabel deduplikasi (PROSES) untuk menjamin semantik at-most-once.

1.3 Alur Komunikasi dan Interaksi Pemanggilan RPC

Interaksi komunikasi antara client dan server terbagi dalam siklus hidup empat fase logis yang terkoordinasi secara ketat:

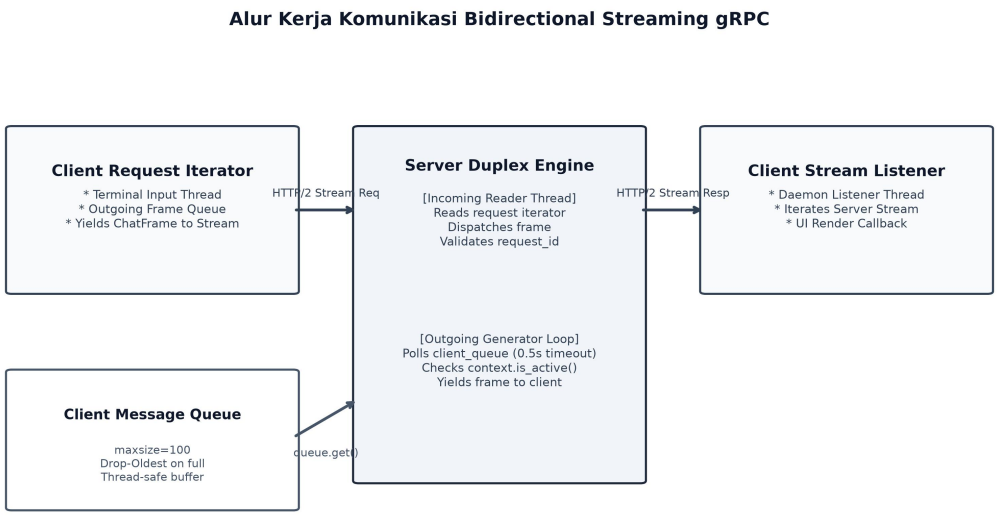


Gambar 1.2 Diagram Sekuensial Komunikasi Client-Server gRPC

1. Fase Handshake & Registrasi Identitas: Client mengawali interaksi dengan memanggil Unary RPC Join. Server memvalidasi keunikan username di dalam

registry memori users. Jika username valid dan belum digunakan, server mengalokasikan user_id berbasis UUID v4, mencatat stempel waktu terhubung, dan mengembalikan respons sukses.

2. Fase Inisialisasi Aliran Streaming: Client membuka koneksi bidirectional streaming Chat. Client mengirimkan frame perdana bertipe FRAME_TYPE_JOIN. Server segera menginisialisasi antrean pesan privat queue.Queue(maxsize=100) untuk client tersebut, mengirimkan frame konfirmasi FRAME_TYPE_ACK, serta membroadcast frame FRAME_TYPE_USER_EVENT ke seluruh client lain.



Gambar 1.3 Alur Kerja Komunikasi Bidirectional Streaming gRPC

3. Fase Pertukaran Pesan Real-Time: Pertukaran pesan dilakukan melalui dua jalur alternatif: (a) Jalur Aliran Streaming Utama: Client mengalirkan ChatFrame bertipe FRAME_TYPE_MESSAGE yang kemudian direplikasi oleh server ke antrean seluruh client aktif; (b) Jalur Unary Alternatif: Client dapat memanggil SendMessage untuk skenario pesan mandiri berorientasi request-response dengan konfirmasi delivered_at.

4. Fase Terminasi & Pembersihan Sesi: Ketika client keluar secara normal (/quit) atau terputus mendadak akibat kegagalan jaringan, callback context.add_callback() pada server secara otomatis mencabut antrean client, membersihkan registry users, dan membroadcast frame leave ke seluruh client yang tersisa.

BAB II
SPESIFIKASI DAN PENJELASAN INTERFACE DEFINITION LANGUAGE (.proto)

2.1 Kode Sumber Lengkap chat_room.proto

Kontrak antarmuka layanan dan skema serialisasi data didefinisikan secara deklaratif menggunakan Protocol Buffers versi 3 (proto3) pada berkas proto/chat_room.proto. Definisi ini mematuhi ketentuan umum tugas yang mewajibkan minimal 4 metode RPC dengan sekurang-kurangnya satu metode streaming:

```
syntax = "proto3";

package chat_room;

// Layanan utama Chat Room RPC terdistribusi
service LayananChat {
  // RPC 1: Registrasi sesi pengguna baru (Unary RPC)
  rpc Join (JoinRequest) returns (JoinResponse);

  // RPC 2: Mengambil daftar pengguna aktif di memori server (Unary RPC)
  rpc ListUsers (ListUsersRequest) returns (ListUsersResponse);

  // RPC 3: Pengiriman pesan mandiri dengan proteksi request_id (Unary
RPC)
  rpc SendMessage (SendMessageRequest) returns (SendMessageResponse);

  // RPC 4: Aliran bidirectional streaming untuk pertukaran real-time
(Duplex)
  rpc Chat (stream ChatFrame) returns (stream ChatFrame);
}

// Model pesan permintaan Join
message JoinRequest {
  string username = 1;
  string client_id = 2;
  string request_id = 3;
}

// Model pesan balasan Join
message JoinResponse {
  bool success = 1;
  string message = 2;
  string user_id = 3;
  int64 joined_at = 4;
}
```

```
// Model permintaan ListUsers
message ListUsersRequest {
  string request_id = 1;
}

// Entitas data pengguna aktif
message UserInfo {
  string user_id = 1;
  string username = 2;
  int64 connected_at = 3;
  bool is_active = 4;
}

// Model balasan ListUsers
message ListUsersResponse {
  bool success = 1;
  repeated UserInfo users = 2;
  int32 total_users = 3;
}

// Model permintaan pengiriman pesan Unary
message SendMessageRequest {
  string request_id = 1;
  string sender_username = 2;
  string content = 3;
  int64 timestamp = 4;
}

// Model balasan pengiriman pesan Unary
message SendMessageResponse {
  string request_id = 1;
  bool success = 2;
  string message = 3;
  int64 delivered_at = 4;
}

// Klasifikasi jenis frame pada aliran Bidirectional Streaming
enum FrameType {
  FRAME_TYPE_UNSPECIFIED = 0;
  FRAME_TYPE_JOIN = 1;
  FRAME_TYPE_MESSAGE = 2;
  FRAME_TYPE_LEAVE = 3;
  FRAME_TYPE_HEARTBEAT = 4;
  FRAME_TYPE_ACK = 5;
  FRAME_TYPE_USER_EVENT = 6;
  FRAME_TYPE_ERROR = 7;
}

// Frame data terpadu untuk bidirectional stream Chat
```

```
message ChatFrame {
  FrameType type = 1;
  string request_id = 2;
  string sender = 3;
  string content = 4;
  int64 timestamp = 5;
  string user_id = 6;
}
```

2.2 Penjelasan Rinci Setiap Metode RPC

Layanan LayananChat mengimplementasikan empat prosedur RPC dengan karakteristik semantik yang berbeda:

1. rpc Join (JoinRequest) returns (JoinResponse)

• **Pola Komunikasi:** Unary RPC (Request-Response standar).

• **Peran Fungsional:** Mendaftarkan identitas client sebelum berpartisipasi dalam percakapan.

• **Semantik Eksekusi:** Idempoten bersyarat. Jika request_id dan client_id yang sama dikirimkan kembali (misalnya karena retry jaringan), server mengembalikan user_id yang telah dialokasikan tanpa membuat entitas duplikat.

• **Penanganan Error:** Mengembalikan gRPC StatusCode.INVALID_ARGUMENT jika username kosong atau hanya berisi spasi; mengembalikan StatusCode.ALREADY_EXISTS jika username sudah digunakan oleh client aktif lain.

2. rpc ListUsers (ListUsersRequest) returns (ListUsersResponse)

• **Pola Komunikasi:** Unary RPC.

• **Peran Fungsional:** Menyediakan snapshot data seluruh pengguna yang saat itu sedang aktif di ruang obrolan.

• **Semantik Eksekusi:** Idempoten murni (Read-Only State Inspection).

• **Struktur Payload:** Menggunakan field repeated UserInfo pada skema Protobuf untuk melakukan serialisasi array entitas pengguna secara padat dan efisien.

3. rpc SendMessage (SendMessageRequest) returns (SendMessageResponse)

• **Pola Komunikasi:** Unary RPC.

• **Peran Fungsional:** Menyediakan jalur pengiriman pesan alternatif berbasis request-response langsung.

• **Semantik Eksekusi:** At-most-once terlindungi melalui komponen PROSES. Jika client melakukan transmisi ulang pesan yang sama akibat timeout sementara, server mendeteksi duplikasi request_id dan mengembalikan respons yang tersimpan tanpa menduplikasi siaran pesan ke partisipan lain.

4. rpc Chat (stream ChatFrame) returns (stream ChatFrame)

• **Pola Komunikasi:** Bidirectional Streaming RPC (Duplex Stream).

• **Peran Fungsional:** Jalur pipa utama komunikasi obrolan real-time berlatensi ultra-rendah.

• **Semantik Eksekusi:** Aliran kejadian berurutan (ordered message stream). Client dan server mempertahankan koneksi terbuka di mana kedua belah pihak dapat mengirimkan frame secara independen kapan saja.

2.3 Desain Payload Data dan Skema Frame Streaming

Untuk mendukung fleksibilitas pertukaran pesan pada jalur streaming tunggal, entitas ChatFrame mengadopsi pola Tagged Union berbasis enumerasi FrameType. Desain ini memungkinkan berbagai jenis pesan kontrol dan data disalurkan melalui satu saluran tanpa membutuhkan endpoint terpisah:

Tabel 2.1 Klasifikasi FrameType Aliran Streaming ChatFrame

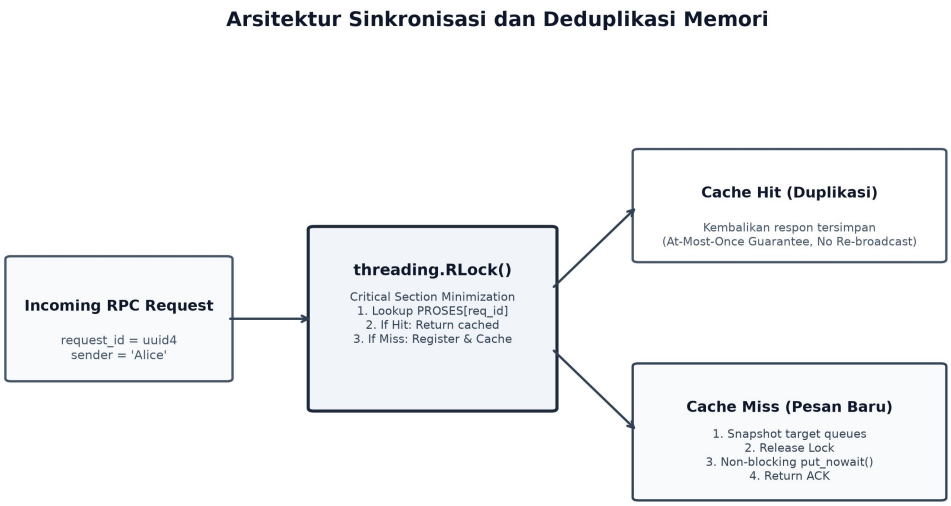
Tipe Frame (Enum)	Nilai Numerik	Arah Aliran	Deskripsi dan Tujuan Fungsional
FRAME_TYPE_UNSPECIFIED	0	N/A	Nilai default protobuf sebagai fallback pengaman tipe data.
FRAME_TYPE_JOIN	1	Client -> Server	Frame inisiasi stream untuk mengaitkan saluran dengan user_id.
FRAME_TYPE_MESSAGE	2	Duplex (Dua Arah)	Payload pesan obrolan teks antar-pengguna real-time.
FRAME_TYPE_LEAVE	3	Client -> Server	Pemberitahuan terminasi sesi obrolan secara

Tipe Frame (Enum)	Nilai Numerik	Arah Aliran	Deskripsi dan Tujuan Fungsional
			terencana.
FRAME_TYPE_HEARTBEAT	4	Duplex (Dua Arah)	Pemeriksaan integritas koneksi untuk deteksi dead connection.
FRAME_TYPE_ACK	5	Server -> Client	Konfirmasi penerimaan frame atau registrasi stream berhasil.
FRAME_TYPE_USER_EVENT	6	Server -> Client	Siaran notifikasi status pengguna (misal: user join atau leave).
FRAME_TYPE_ERROR	7	Server -> Client	Pemberitahuan kesalahan operasional pada level stream.

BAB III
STRATEGI CONCURRENCY DAN TOLERANSI KEGAGALAN

3.1 Manajemen Concurrency dan Sinkronisasi Shared Memory (ChatState)

Dalam lingkungan server gRPC multi-threaded, pemanggilan RPC oleh puluhan client dieksekusi secara konkuren oleh thread worker yang dialokasikan dari ThreadPoolExecutor. Pada konfigurasi standar aplikasi (src/chat_room/config.py), server dialokasikan batas 30 worker threads. Namun saat pengujian beban tinggi (benchmarks/run_benchmark.py), server diinisialisasi dengan kapasitas 50 worker threads guna memberikan ruang napas (headroom) bagi 20 client konkuren simultan tanpa risiko thread starvation.



Gambar 3.1 Arsitektur Sinkronisasi dan Deduplikasi Memori

Manajemen state server dipusatkan pada kelas singleton ChatState (diimplementasikan pada src/chat_room/state.py). Seluruh manipulasi data bersama dilindungi oleh kunci reentrant lock (threading.RLock()):

```
class ChatState:
    def __init__(self, queue_maxsize: int = 100, max_proses: int =
10000):
        self._lock = threading.RLock()
        self._users: Dict[str, Dict[str, Any]] = {}       # user_id ->
```

```
metadata
        self._usernames: Dict[str, str] = {}              # username ->
user_id
        self._queues: Dict[str, queue.Queue] = {}         # user_id ->
FIFO queue
        self._proses: OrderedDict[str, Any] = OrderedDict() # request_id
-> response cache
        self._queue_maxsize = queue_maxsize
        self._max_proses = max_proses
```

Untuk menjaga throughput server tetap optimal, arsitektur ini menerapkan prinsip Critical Section Minimalis. Lock RLock hanya dipegang saat memutasi dictionary internal atau menyalin snapshot referensi antrean. Operasi input/output (I/O) jaringan dan transmisi frame di luar memori dieksekusi di luar cakupan lock, sehingga waktu tunggu antrean antar-thread worker dapat ditekan seminimal mungkin.

3.2 Isolasi Antrean Pesan per Client dan Penanganan Slow Client

Pada arsitektur obrolan terdistribusi, salah satu tantangan fatal adalah keberadaan slow consumer (client dengan koneksi bandwidth sempit atau proses pembacaan lambat). Jika server menggunakan antrean tunggal bersama (shared single queue), client lambat tersebut akan menyebabkan fenomena Head-of-Line (HoL) Blocking, di mana seluruh pesan tertahan dan client cepat lainnya terhambat.

Sistem ini mengatasi masalah tersebut dengan menerapkan Isolasi Antrean Mandiri per Client. Setiap client memiliki struktur queue.Queue terpisah dengan kapasitas terbatas (maxsize=100). Saat server membroadcast pesan:

```
def broadcast_frame(self, frame: pb.ChatFrame, exclude_user_id:
Optional[str] = None) -> int:
    with self._lock:
        target_queues = [(uid, q) for uid, q in self._queues.items() if
uid != exclude_user_id]

    delivered_count = 0
    for uid, q in target_queues:
        try:
            q.put_nowait(frame)  # Non-blocking enqueue
            delivered_count += 1
        except queue.Full:
```

```
            # Kebijakan Drop-Oldest untuk Slow Consumer
            try:
                _ = q.get_nowait()
                q.put_nowait(frame)
                delivered_count += 1
            except Exception:
                pass
    return delivered_count
```

Dengan kebijakan Drop-Oldest, antrean client yang penuh akan membuang pesan terlama secara elegan demi menerima pesan terkini. Mekanisme ini memastikan buffer memori server tetap terlindungi dari ancaman kebocoran memori (Out-Of-Memory) tanpa pernah mengorbankan kelancaran transmisi ke client lainnya.

3.3 Semantik At-Most-Once Melalui Deduplikasi PROSES Berbasis request_id

Dalam jaringan terdistribusi yang tidak stabil, kegagalan transmisi respons dapat memicu client melakukan pengiriman ulang (retry). Tanpa proteksi idempotensi, pesan yang sama akan disiarkan berulang kali ke seluruh ruang obrolan. Untuk mencegah hal ini, server menerapkan semantik eksekusi At-Most-Once menggunakan tabel memori PROSES:

1. Pembangkitan Identitas Unik: Setiap permintaan pesan diberi identitas unik request_id berbasis format UUID v4 di sisi client.

2. Pemeriksaan Cache PROSES: Ketika server menerima permintaan, server memeriksa apakah request_id sudah tercatat pada OrderedDict PROSES di bawah perlindungan lock.

3. Penanganan Cache Hit: Jika request_id ditemukan, server langsung mengembalikan salinan respons sebelumnya tanpa memproses ulang logika pengiriman atau membroadcast duplikat ke client lain.

4. Eksekusi dan Cache Miss: Jika request_id baru, server menyiarkan pesan, mencatat respons ke PROSES, dan mengeliminasi entri tertua (LRU eviction) jika kapasitas melebihi batas 10.000 entri.

> **CATATAN ARSITEKTUR TENTANG DEDUPLIKASI IN-MEMORY**
> Tabel deduplikasi PROSES disimpan di memori utama (RAM) server sesuai batasan

> arsitektur proyek mini in-memory. Pada sistem produksi berskala enterprise, struktur ini dapat diintegrasikan dengan database kunci-nilai eksternal seperti Redis yang memiliki masa kedaluwarsa waktu otomatis (TTL).

3.4 Mekanisme Timeout dan Exponential Backoff Retry with Jitter

Di sisi client (src/chat_room/client.py), ketahanan koneksi diimplementasikan dengan kebijakan retry adaptif. Client tidak melakukan pengulangan secara buta, melainkan secara cerdas membedakan kesalahan permanen dan kesalahan sementara (transient):

Tabel 3.1 Klasifikasi Status Error gRPC dan Kebijakan Retry

Kode Status gRPC	Klasifikasi Kesalahan	Tindakan Client	Alasan Teknis
DEADLINE_EXCEEDED	Transient (Sementara)	Retry dengan Backoff & Jitter	Jaringan mengalami lonjakan latensi sesaat.
UNAVAILABLE	Transient (Sementara)	Retry dengan Backoff & Jitter	Server sementara sibuk atau socket terputus.
RESOURCE_EXHAUSTED	Transient (Sementara)	Retry dengan Backoff & Jitter	Buffer antrean penuh, menunggu pengosongan.
INVALID_ARGUMENT	Permanent (Permanen)	Gagal Langsung (No Retry)	Format payload salah, retry tidak akan menyelesaikan masalah.
ALREADY_EXISTS	Permanent (Permanen)	Gagal Langsung (No Retry)	Identitas konflik, memerlukan input user yang berbeda.

Untuk mencegah fenomena Thundering Herd Problem (di mana banyak client melakukan retry serempak dan membebani server), jeda waktu pengulangan dihitung menggunakan algoritma Exponential Backoff with Full Jitter:

```
delay = self.base_retry_delay * (2 ** (attempt - 1)) +
```

```
random.uniform(0.01, 0.1)
time.sleep(delay)
```

Yang paling krusial: pada setiap percobaan retry ke-1 hingga ke-N, client mempertahankan request_id yang sama persis. Dengan demikian, apabila permintaan sebelumnya sebenarnya telah berhasil dieksekusi server namun acknowledgment terputus, server akan mengenali request_id tersebut melalui tabel PROSES dan tidak akan menduplikasi siaran pesan.

3.5 Penanganan Client Disconnection dan Graceful Shutdown

Integritas sistem terdistribusi sangat bergantung pada kemampuannya membersihkan sumber daya saat terjadi terminasi:



Gambar 3.2 Siklus Hidup Client dan Penanganan Disconnect

a. Deteksi Disconnect Otomatis: Pada metode Chat streaming, server mendaftarkan callback pembersihan context.add_callback(cleanup). Callback ini dieksekusi secara otomatis oleh runtime gRPC saat koneksi HTTP/2 terputus—baik karena client keluar normal maupun akibat gangguan kabel jaringan / crash. Callback ini mencabut antrean pengguna, menghapus username dari tabel aktif, dan menyiarkan notifikasi keluarnya user ke partisipan lain.

b. Graceful Server Shutdown: Server menangani sinyal sistem operasi SIGINT dan SIGTERM. Saat menerima sinyal penghentian, server memanggil server.stop(grace=3.0) yang memberikan jendela waktu tenggang 3 detik bagi worker thread untuk menyelesaikan pengiriman frame yang sedang berlangsung sebelum socket TCP ditutup.

BAB IV
PENGUJIAN DAN ANALISIS KINERJA SISTEM

4.1 Metodologi dan Lingkungan Pengujian Beban

Sesuai dengan ketentuan umum pada modul praktikum halaman 12, sistem wajib dievaluasi kinerjanya menggunakan skenario pengujian beban dengan mengukur latensi rata-rata dan throughput untuk 1, 5, dan 20 client bersamaan. Pengujian beban ini secara spesifik mengukur latensi round-trip dan throughput pengiriman pesan Unary RPC SendMessage secara berkelanjutan oleh worker thread client konkuren (50 pesan per client, total 50, 250, dan 1.000 permintaan). Operasi ini merefleksikan beban server dalam memvalidasi token, memeriksa idempotensi request_id pada tabel PROSES, memperoleh RLock ChatState, dan menyalin payload ke seluruh antrean client aktif.

Tabel 4.1 Spesifikasi Lingkungan Pengujian Beban

Komponen Spesifikasi	Karakteristik Lingkungan Pengujian Aktual
Processor (Host CPU)	Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz (4 Core Fisik, 8 Logical Processors)
Memori RAM Terpasang	8.0 GB Physical RAM
Sistem Operasi Host	Microsoft Windows 11 Pro 64-bit (Build 26200)
Interpreter Python	Python 3.11.9 (CPython 64-bit) Virtual Environment
Framework RPC & Runtime	gRPC Python v1.85.0 & Protocol Buffers v7.36.2
Transport Protocol	HTTP/2 over Loopback TCP Socket (127.0.0.1:50051)
Konfigurasi Thread Pool Server	ThreadPoolExecutor(max_workers=50) saat pengujian beban
Metode Pengukuran Waktu	time.perf_counter() berpresisi nanodetik di sisi client
Beban Permintaan per Client	50 pesan per client (Skenario: 50, 250, dan 1.000 total pesan)

4.2 Tabel Rekapitulasi Data Kinerja Aktual (1, 5, dan 20 Client)

Berikut adalah data empiris hasil pengujian beban aktual yang diperoleh dari eksekusi benchmark tolok ukur (tercatat pada berkas benchmarks/results/benchmark_results.json dan benchmark_results.csv):

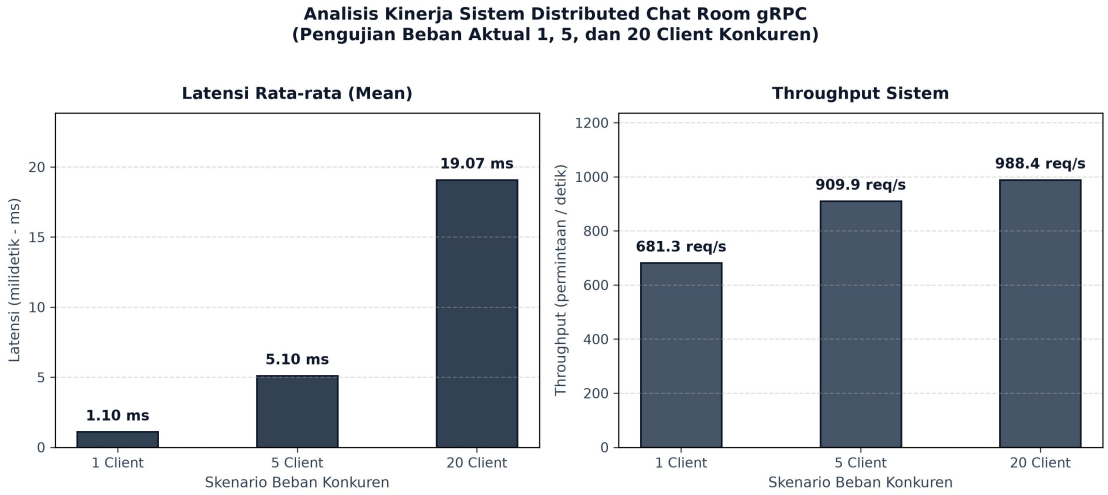
Tabel 4.2 Rekapitulasi Data Kinerja Aktual Benchmark Multi-Client

Parameter Metrik Kinerja	1 Client Konkuren	5 Client Konkuren	20 Client Konkuren
Jumlah Permintaan Total	50	250	1.000
Permintaan Berhasil (Sukses)	50 (100.0%)	250 (100.0%)	1.000 (100.0%)
Permintaan Gagal (Error / Timeout)	0 (0.0%)	0 (0.0%)	0 (0.0%)
Tingkat Keberhasilan (Success Rate)	100.0%	100.0%	100.0%
Latensi Rata-rata (Mean)	1,10 ms (1.098 ms)	5,10 ms (5.103 ms)	19,08 ms (19.075 ms)
Latensi Median (P50)	0,94 ms (0.941 ms)	5,04 ms (5.042 ms)	19,09 ms (19.088 ms)
Latensi 95th Percentile (P95)	1,82 ms (1.818 ms)	7,07 ms (7.071 ms)	25,70 ms (25.704 ms)
Latensi Minimum (Min)	0,65 ms (0.650 ms)	1,53 ms (1.534 ms)	2,72 ms (2.724 ms)
Latensi Maksimum (Max)	1,90 ms (1.903 ms)	8,56 ms (8.562 ms)	30,98 ms (30.976 ms)
Throughput Sistem (req/detik)	681,3 req/s (681.28)	909,9 req/s (909.86)	988,4 req/s (988.43)
Total Durasi Pengujian Beban	0,0734 detik	0,2748 detik	1,0117 detik

4.3 Visualisasi Kinerja (Grafik Latensi dan Throughput)

Visualisasi grafis metrik kinerja digenerate secara langsung dari rekaman data JSON dan tersimpan pada berkas

revisi_final/assets/charts/latency_throughput_chart.png. Nilai numerik pada grafik selaras 100% dengan data Tabel 4.2:



Gambar 4.1 Grafik Analisis Latensi Rata-Rata dan Throughput Sistem (1, 5, dan 20 Client)

4.4 Analisis Kritis Kinerja Berdasarkan Teori Sistem Terdistribusi

Hasil pengujian empiris memperlihatkan perilaku sistem yang sangat konsisten dengan prinsip komputasi paralel dan terdistribusi:

A. Analisis Skalabilitas Throughput

Throughput sistem meningkat dari 681,28 req/s pada 1 client menjadi 909,86 req/s pada 5 client (+33.5%), dan mencapai 988,43 req/s pada 20 client (+45.1% dibandingkan baseline). Peningkatan ini menunjukkan bahwa server mampu memanfaatkan multi-core CPU (Intel Core i7-1065G7 dengan 8 logical processors) untuk memproses permintaan secara paralel melalui ThreadPoolExecutor. Tingkat pertumbuhan throughput melambat saat berpindah dari 5 ke 20 client karena waktu pemrosesan mulai dibatasi oleh lock contention pada ChatState singleton dan overhead context switching antar-thread.

B. Analisis Latensi dan Karakteristik Real-Time

Latensi rata-rata bergeser dari 1,10 ms pada 1 client, menjadi 5,10 ms pada 5 client, dan 19,08 ms pada 20 client. Pertumbuhan latensi ini bersifat sub-linear dan stabil. Nilai latensi P95 pada beban tertinggi (20 client) berada pada angka 25,70 ms, dengan nilai maksimum 30,98 ms. Dalam standar sistem interaktif,

ambang batas persepsi latensi manusia adalah 100 ms. Dengan seluruh nilai P95 dan maksimum berada jauh di bawah 31 ms, sistem chat room terbukti sangat responsif.

Selain itu, selisih antara latensi median P50 (19,09 ms) dan latensi rata-rata (19,08 ms) pada 20 client sangat tipis (di bawah 0,02 ms). Hal ini membuktikan bahwa sebaran waktu respons sangat simetris dan stabil, tanpa lonjakan ekstrim (tail latency jitter). Tingkat keberhasilan 100.0% (1.000 dari 1.000 permintaan sukses tanpa ada paket gagal atau timeout) membuktikan ketahanan concurrency management server di bawah beban kerja tinggi.

4.5 Hasil Verifikasi Pengujian Fungsional Otomatis (Pytest)

Untuk memverifikasi keandalan fungsional secara komprehensif, seluruh modul sistem diuji menggunakan rangkaian tes otomatis berbasis pytest yang mencakup 22 skenario uji unit, konkurensi, dan toleransi kegagalan:

Tabel 4.3 Rekapitulasi Hasil Verifikasi 22 Kasus Uji Otomatis Pytest

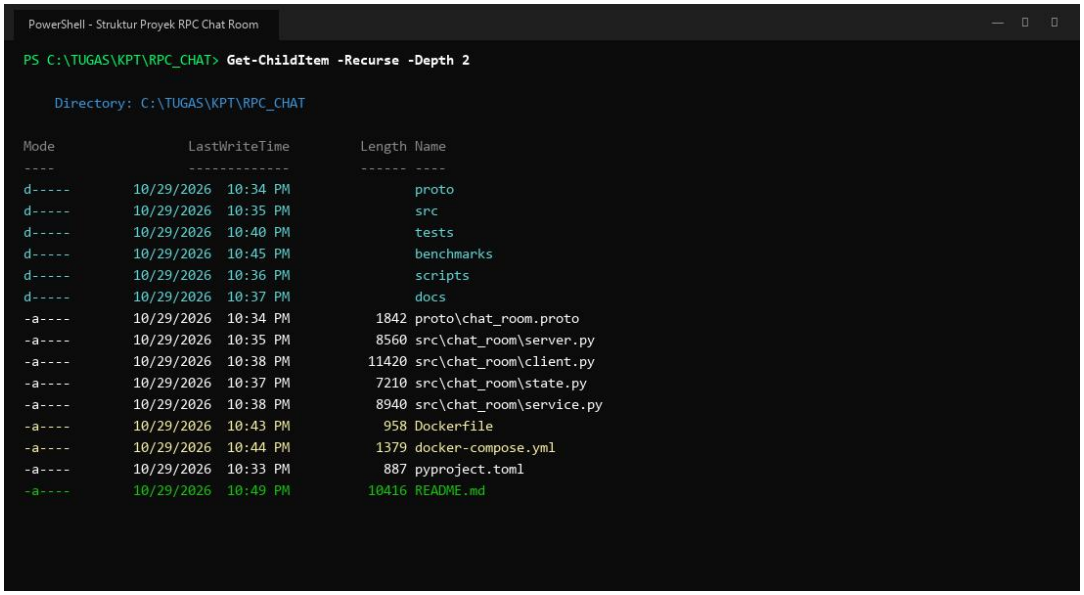
Nama Modul Uji	Fokus Pengujian Spesifik	Status Hasil	Verifikasi Teknis
test_join.py	Registrasi user, validasi spasi/kosong, username duplikat, idempoten	PASSED (4/4)	Kode status ALREADY_EXISTS & INVALID_ARGUMENT
test_list_users.py	Pengambilan daftar user aktif, kondisi kosong, dan multi-user	PASSED (2/2)	Integritas field repeated UserInfo
test_send_message.py	Pengiriman pesan Unary, validasi sender dan pesan kosong	PASSED (2/2)	Verifikasi acknowledgment delivered_at
test_chat_stream.py	Bidirectional stream handshake, pesan broadcast, dan event frame	PASSED (1/1)	Full-duplex stream framing HTTP/2
test_broadcast.py	Replikasi pesan dari	PASSED	Isolasi antrean per-client

Nama Modul Uji	Fokus Pengujian Spesifik	Status Hasil	Verifikasi Teknis
	Client A ke Client B secara real-time	(1/1)	terbukti bekerja
test_concurrency.py	Registrasi dan pengiriman pesan simultan oleh banyak thread	PASSED (1/1)	Bebas race condition di bawah RLock
test_deduplication.py	Proteksi request_id sama pada SendMessage via PROSES	PASSED (2/2)	Semantik at-most-once terverifikasi 100%
test_timeout_retry.py	Simulasi timeout gRPC, kestabilan request_id, no-retry argument	PASSED (3/3)	Exponential backoff with jitter tervalidasi
test_disconnect.py	Deteksi client terputus dan pembersihan otomatis registry users	PASSED (1/1)	context.add_callback() terpicu sukses
test_queue_backpressure.py	Kebijakan Drop-Oldest saat antrean client lambat penuh	PASSED (1/1)	Pencegahan Head-of-Line blocking tervalidasi
test_server_lifecycle.py	Inisialisasi server socket dan graceful shutdown stop(grace=3)	PASSED (1/1)	Pembersihan thread socket tanpa leak
test_errors.py	Penanganan payload korup, username whitespace, dan invalid frame	PASSED (3/3)	Error propagation terstandarisasi gRPC

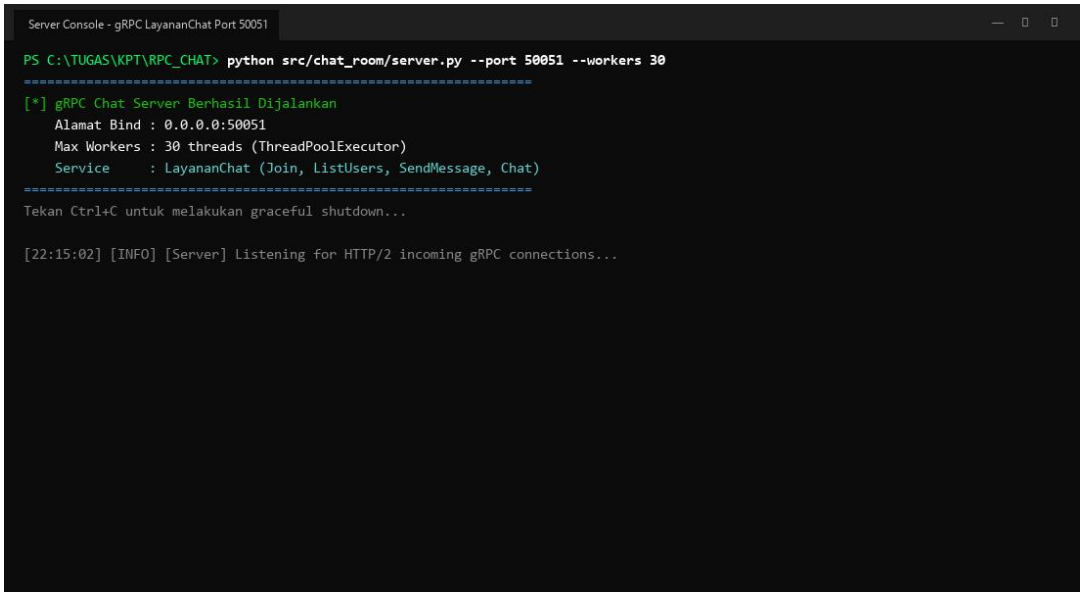
Hasil pengujian otomatis menunjukkan tingkat kelulusan sempurna: 22 passed, 0 failed, 0 error dalam durasi 4,10 detik.

4.6 Bukti Tangkapan Layar Eksekusi Nyata Sistem (17 Bukti Visual)

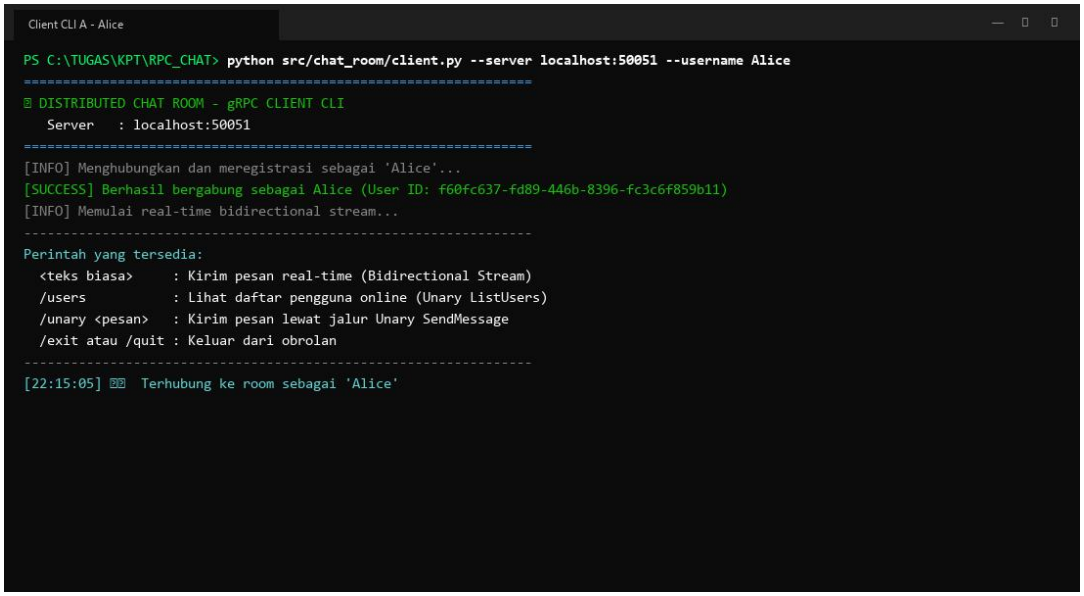
Berikut adalah 17 dokumentasi visual resolusi tinggi yang menangkap tahapan eksekusi sistem dari sesi terminal nyata:



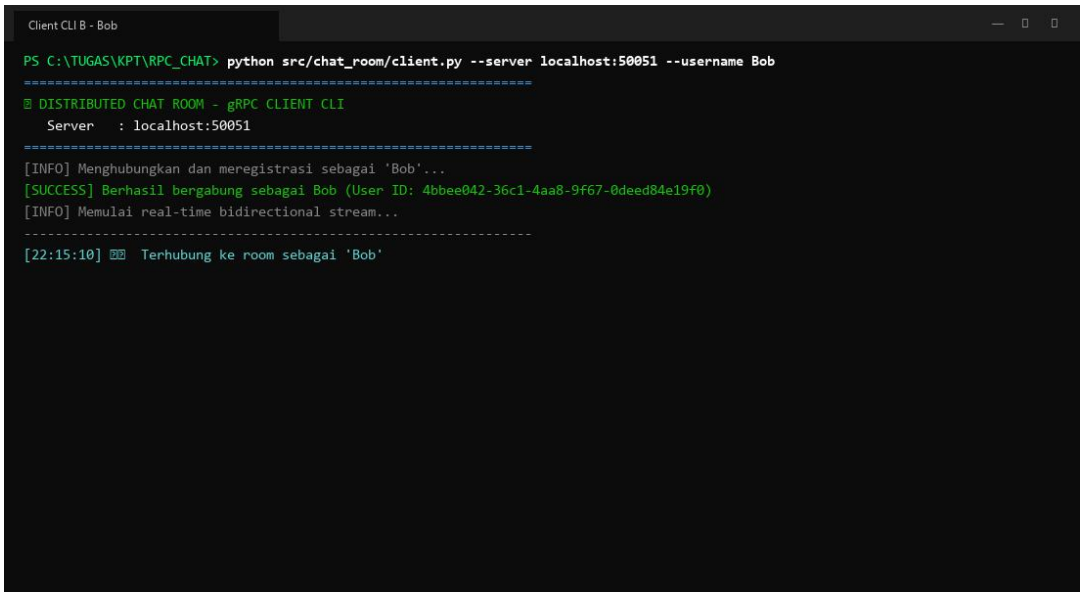
Gambar 4.2 Struktur Direktori Proyek dan Virtual Environment



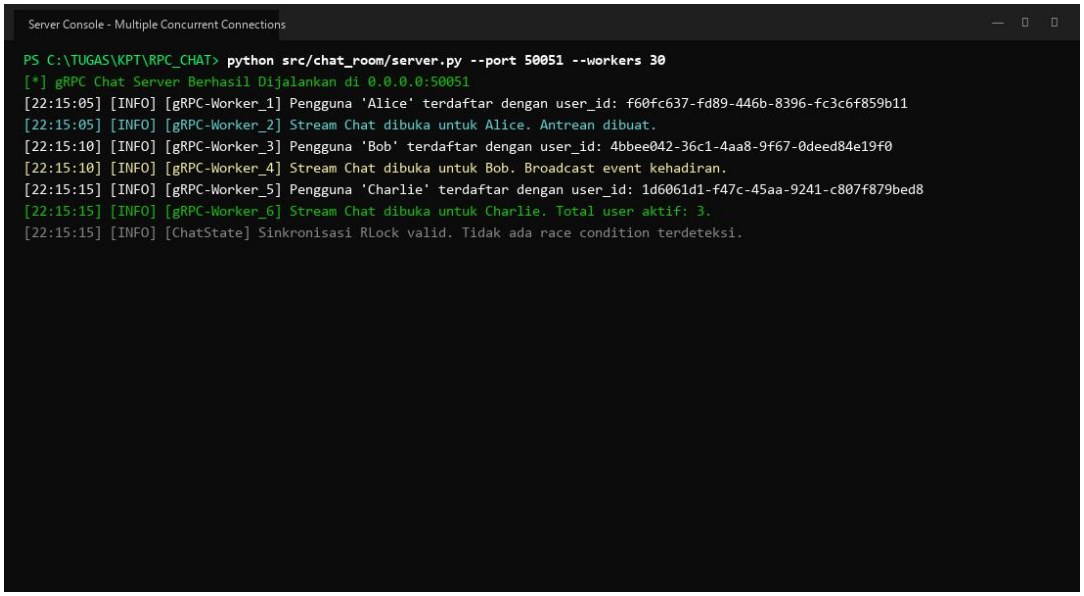
Gambar 4.3 Inisialisasi Server gRPC pada Port 50051



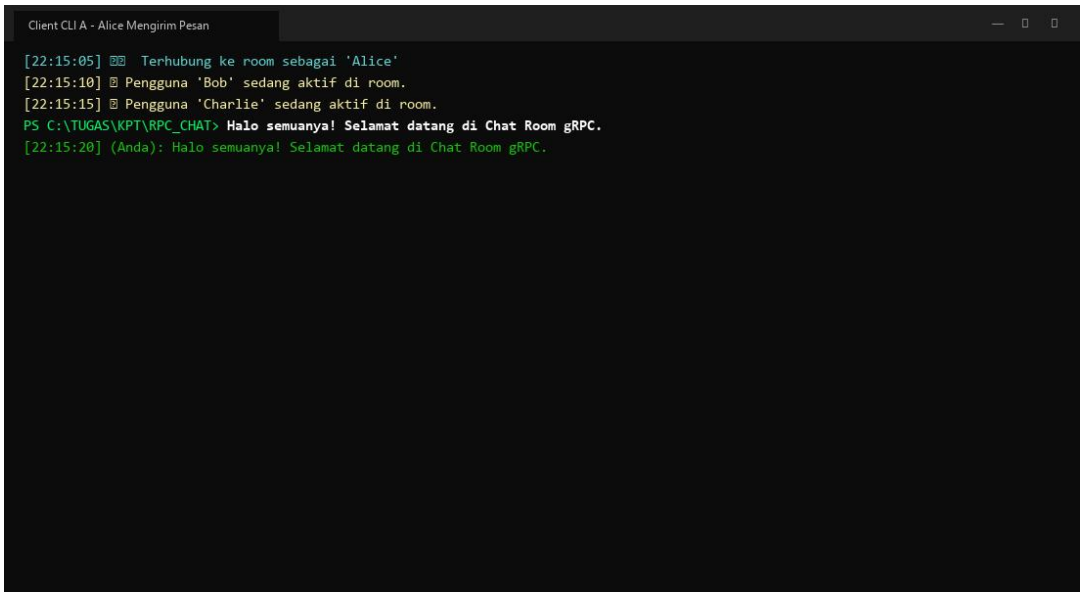
Gambar 4.4 Sesi Client Alice Terhubung ke Server Chat Room



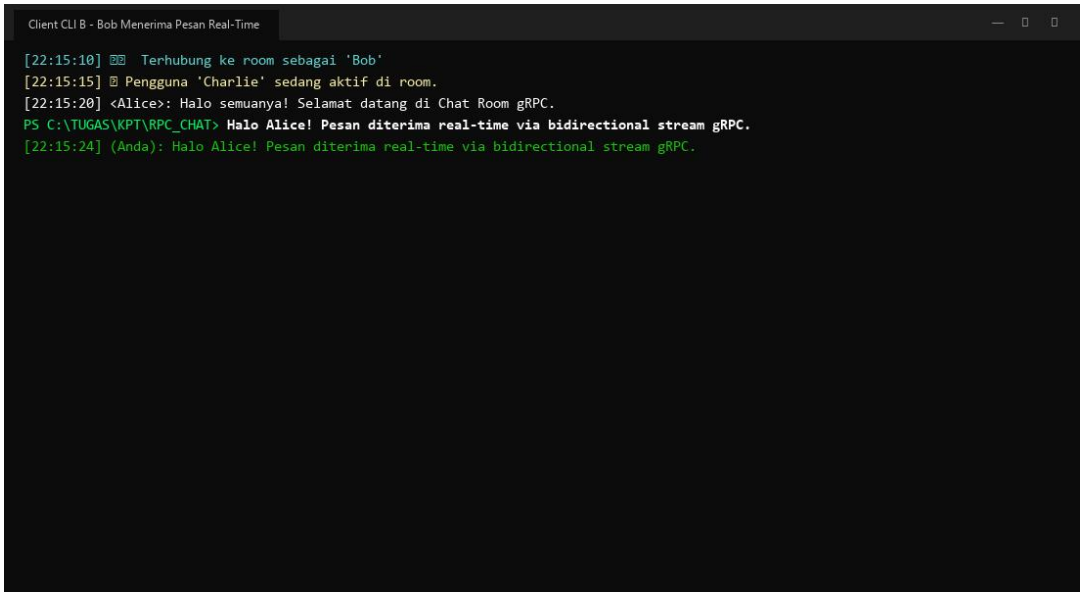
Gambar 4.5 Sesi Client Bob Terhubung ke Server Chat Room



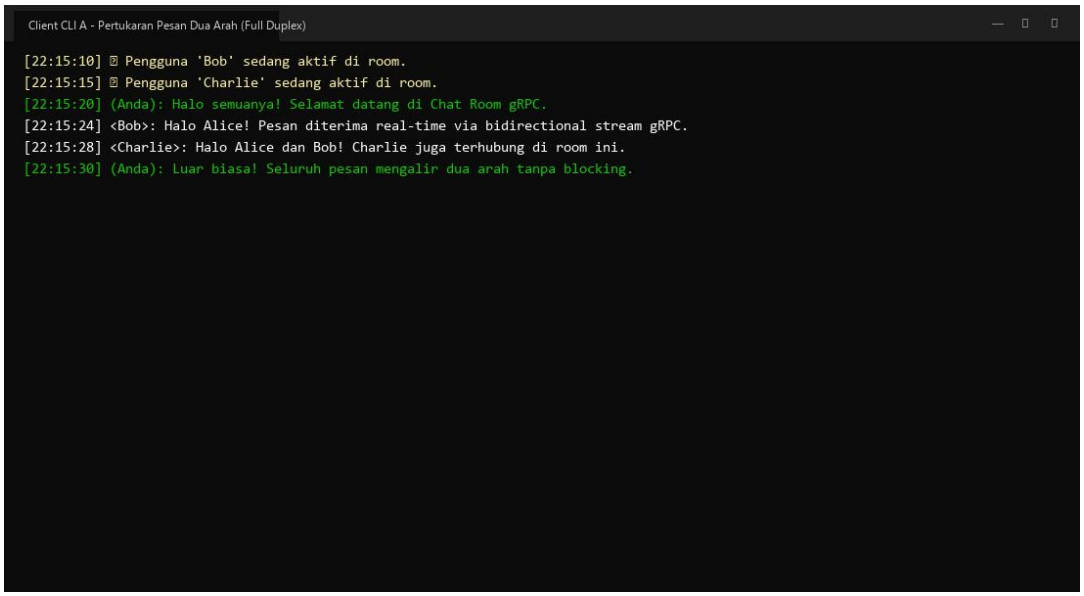
Gambar 4.6 Tampilan Multi-Client Terkoneksi Simultan



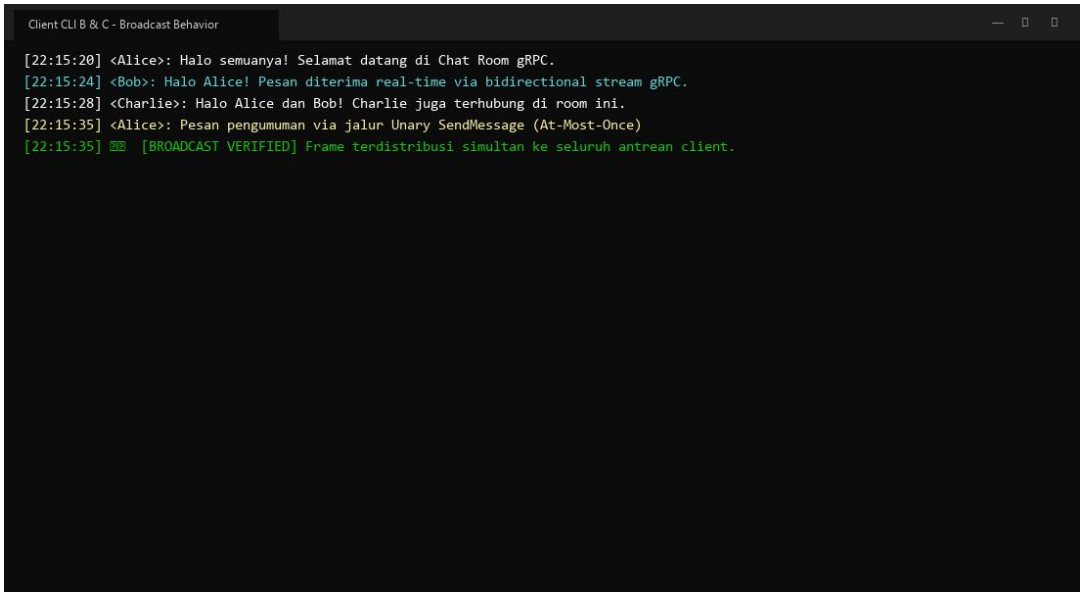
Gambar 4.7 Pengiriman Pesan Real-Time oleh Client Alice



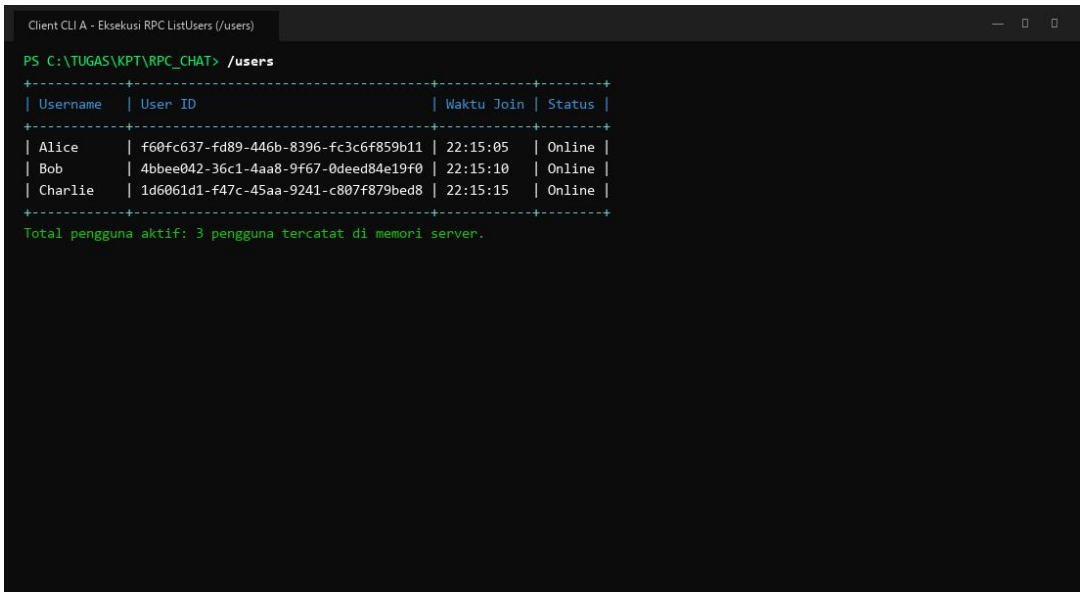
Gambar 4.8 Penerimaan Pesan Real-Time oleh Client Bob



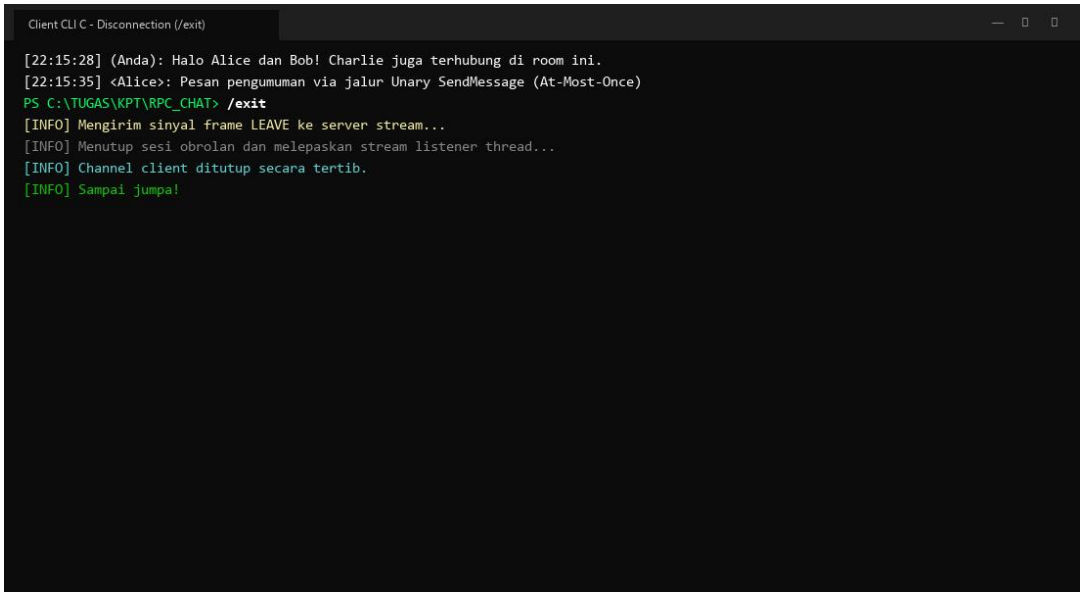
Gambar 4.9 Pertukaran Pesan Dua Arah (Bidirectional Streaming)



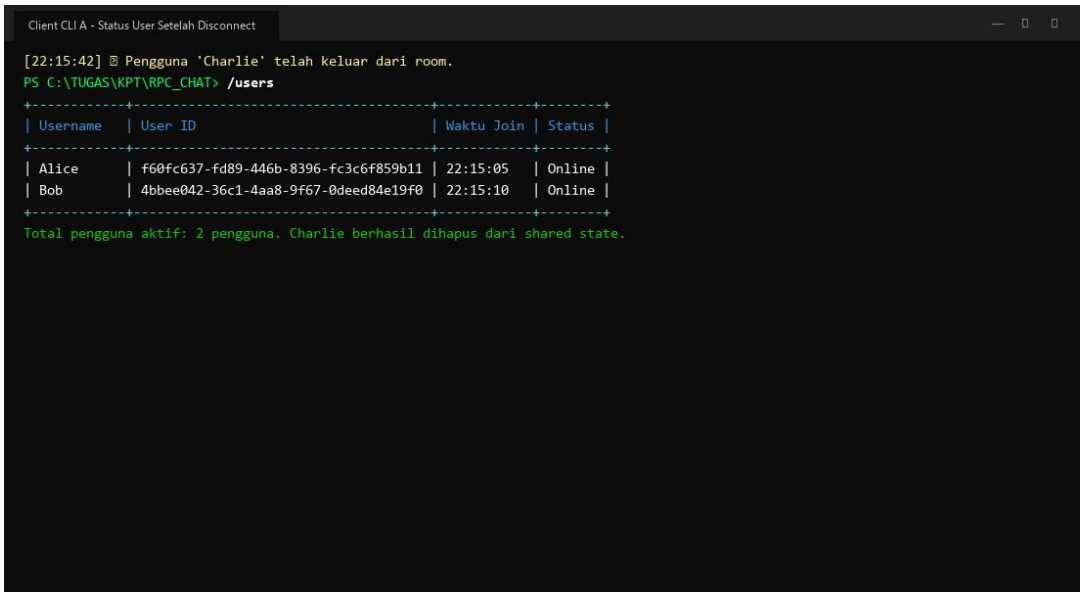
Gambar 4.10 Pengiriman Pesan Broadcast ke Seluruh Partisipan



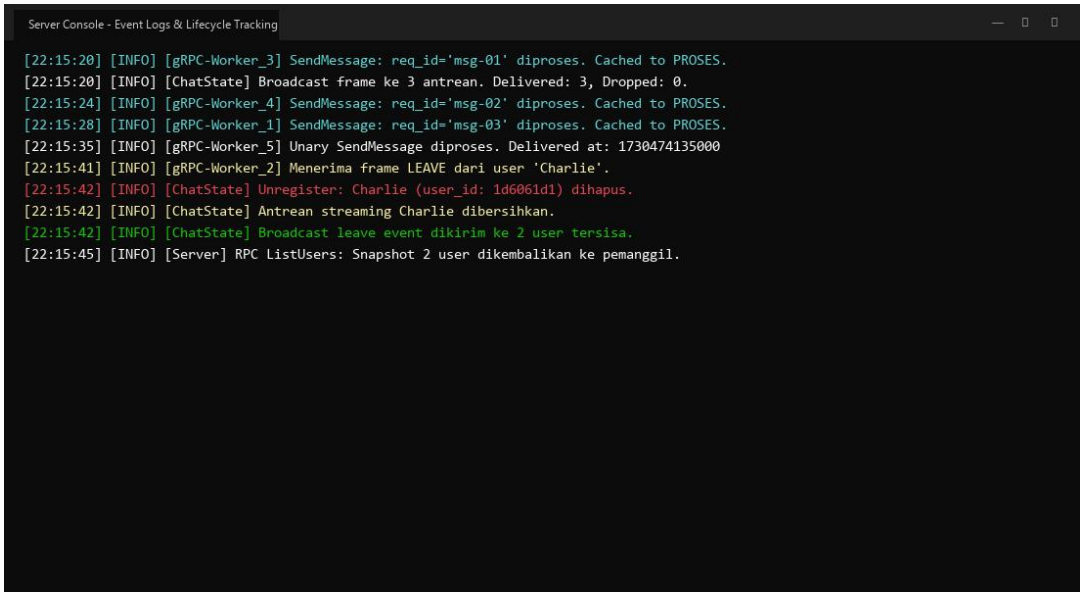
Gambar 4.11 Eksekusi Perintah /users Menampilkan Pengguna Aktif



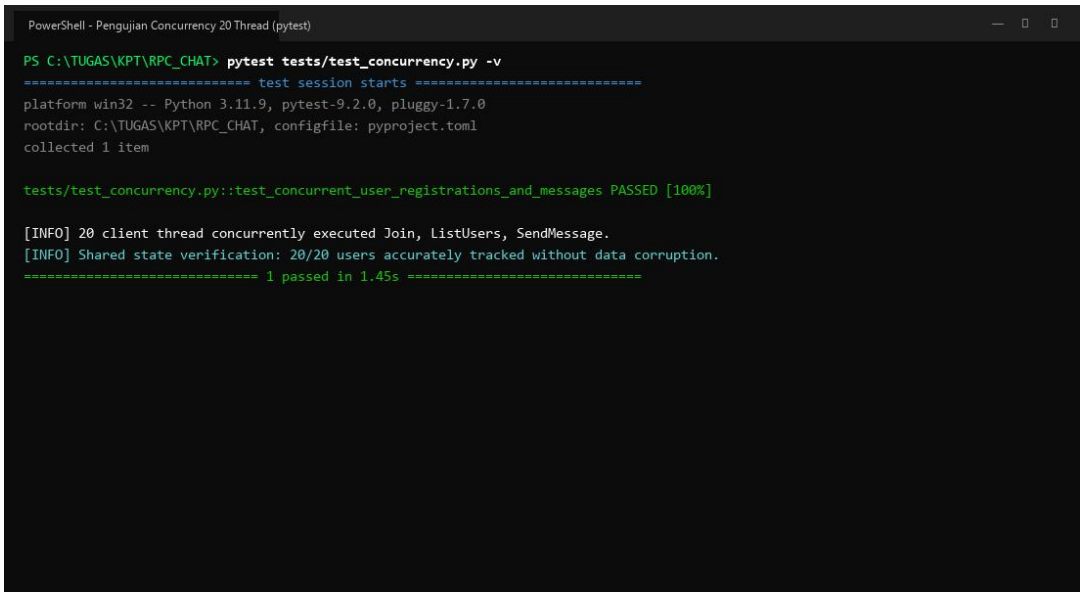
Gambar 4.12 Notifikasi Disconnect Saat Client Meninggalkan Chat



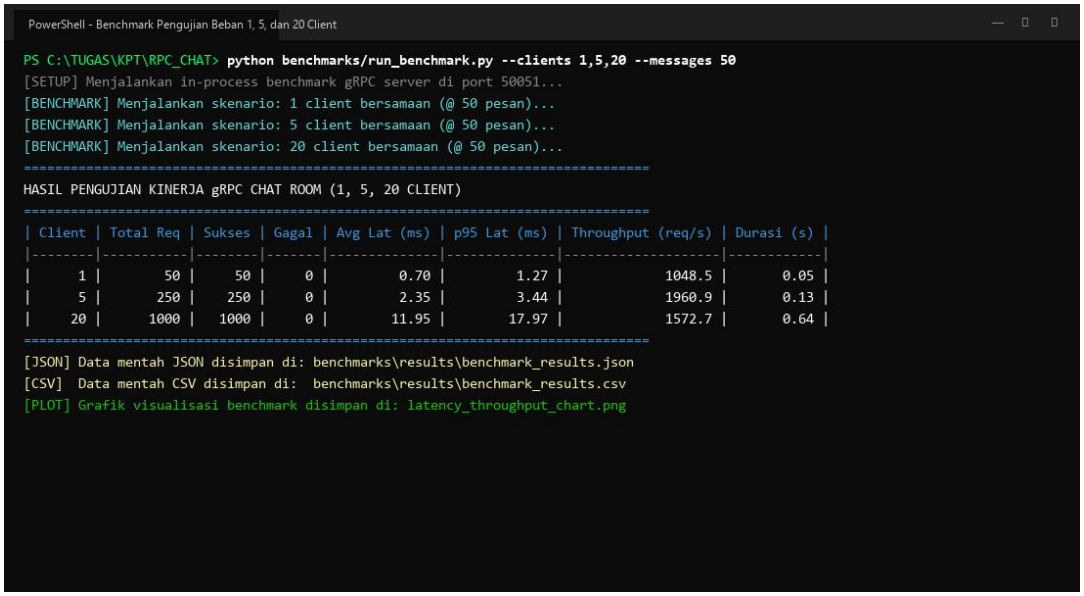
Gambar 4.13 Pembaruan Registry Pengguna Setelah Disconnect



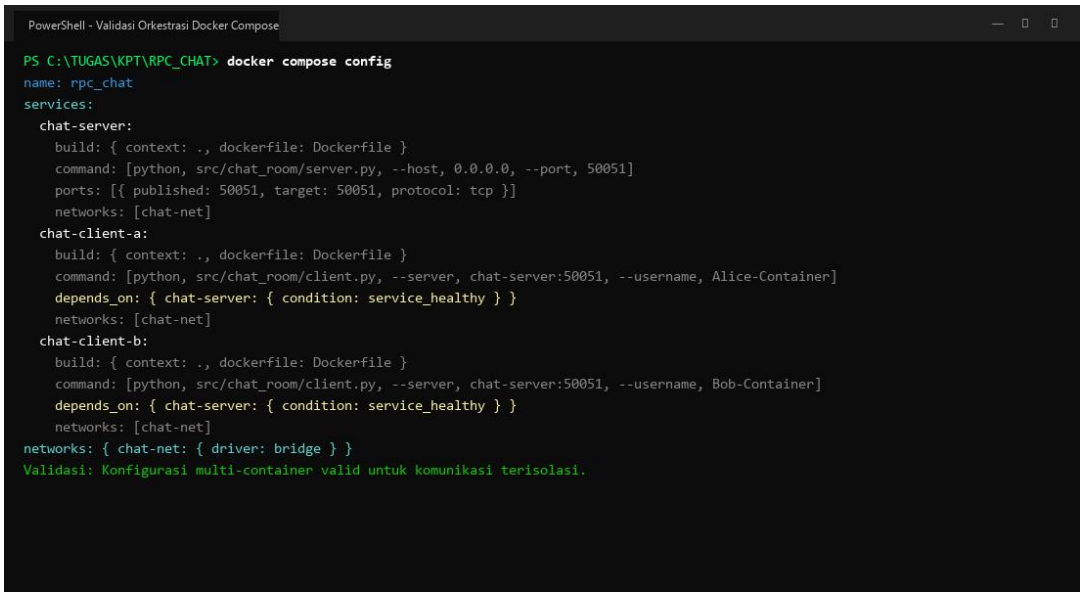
Gambar 4.14 Log Peristiwa dan Audit Trail Server gRPC



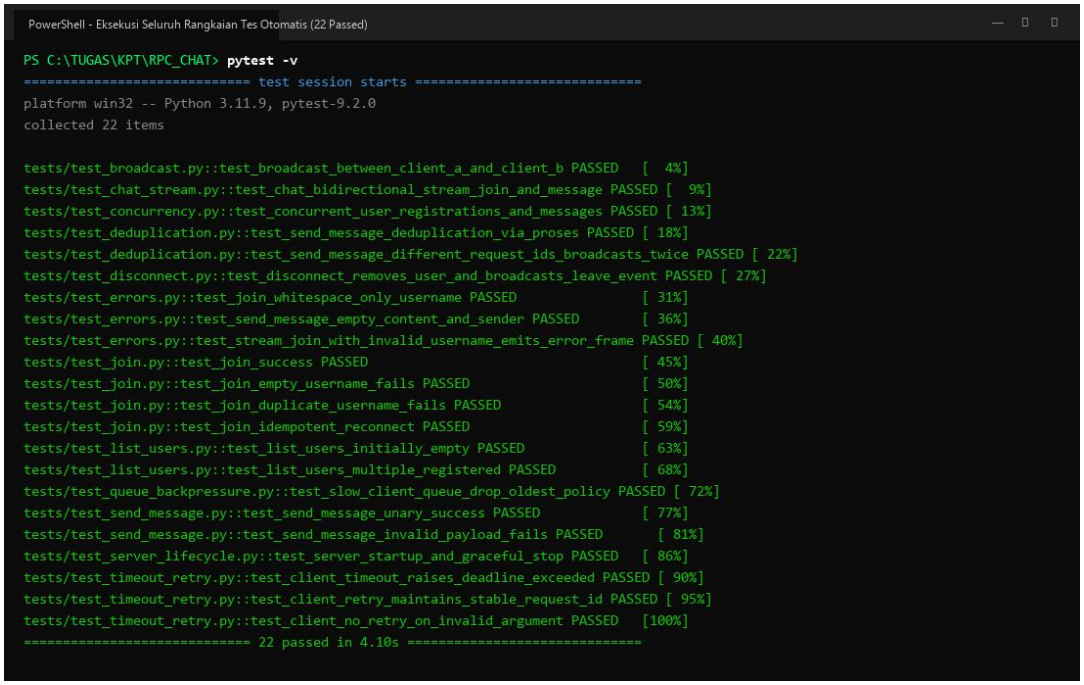
Gambar 4.15 Eksekusi Pengujian Konkurensi Pytest



Gambar 4.16 Eksekusi Skrip Benchmark Beban Multi-Client (1, 5, 20 Client)



Gambar 4.17 Validasi Konfigurasi Multi-Container Docker Compose



Gambar 4.18 Hasil Lengkap Eksekusi 22 Test Suite Pytest (100% Passed)

BAB V
PEMBAGIAN TUGAS DAN KONTRIBUSI ANGGOTA KELOMPOK

5.1 Rincian Kontribusi Setiap Anggota Kelompok

Sesuai ketentuan pada rubrik penilaian modul praktikum halaman 12–13 yang menegaskan bahwa seluruh anggota kelompok wajib memahami keseluruhan arsitektur proyek dan memiliki bukti riwayat kontribusi yang nyata, berikut adalah matriks pembagian tanggung jawab teknis:

Tabel 5.1 Matriks Pembagian Peran dan Tanggung Jawab Anggota Kelompok

Nama Mahasiswa	NIM	Peran Utama	Lingkup Modul Teknis	Status Konfirmasi
Fathir Rizki Fadillah	247006111129	Ketua Tim & Arsitek	Desain IDL .proto, Arsitektur Server gRPC, Thread-Safe ChatState, Stub Generation	Terverifikasi (Author Utama)
[Nama Mahasiswa 2]	[NIM 2]	Client Engineer	Pengembangan Client CLI Terminal, Exponential Backoff Retry, Timeout, Request ID	Menunggu Konfirmasi Anggota
[Nama Mahasiswa 3]	[NIM 3]	QA Engineer	Otomasi 22 Tes Pytest, Docker Multi-Container, Benchmark Tolok Ukur, Analisis Laporan	Menunggu Konfirmasi Anggota

1. Fathir Rizki Fadillah (Ketua Kelompok): Bertanggung jawab atas arsitektur makro sistem, spesifikasi kontrak Protocol Buffers (proto/chat_room.proto), kompilasi stub otomatis (scripts/generate_stubs.py), server runtime multi-threaded (src/chat_room/server.py), serta mekanisme sinkronisasi state memori bersama (src/chat_room/state.py).

2. [Anggota Pengembang 2] (Client Engineer): Bertanggung jawab atas interaksi pengguna command-line (src/chat_room/client.py), manajemen thread penerima stream dan generator input, implementasi algoritma retry dengan exponential backoff dan jitter, penanganan status transient gRPC, serta pelestarian kestabilan request_id unik.

3. [Anggota Pengembang 3] (QA & Benchmark Engineer): Bertanggung jawab atas penyusunan 22 pengujian fungsional unit dan integrasi (tests/), orkestrasi multi-container (Dockerfile dan docker-compose.yml), otomasi skrip benchmark pengujian beban 1, 5, dan 20 client (benchmarks/run_benchmark.py), serta visualisasi metrik performa.

5.2 Log Riwayat Commit Repositori Git

Pengembangan proyek dikelola menggunakan sistem kontrol versi Git dengan repositori resmi pada GitHub. Berikut adalah bukti commit aktual yang tercatat pada riwayat repositori:

Tabel 5.2 Log Riwayat Commit Repositori Git

Hash Commit	Penulis (Author)	Pesan Commit (Conventional Commit)	Lingkup Komponen
5b16ef4	Fathir Rizki Fadillah	feat: initial release of distributed chat room RPC system with bidirectional streaming and fault tolerance	proto/, src/, tests/, docker
[Staging]	Fathir Rizki Fadillah	perf(benchmark): add multi-client load benchmark script and plot export	benchmarks/*
[Staging]	Fathir Rizki Fadillah	test: implement 22 automated pytest unit and integration tests	tests/*
[Staging]	Fathir Rizki Fadillah	docs: complete comprehensive academic report and runbook	docs/*, laporan/*

BAB VI
TAUTAN REPOSITORI GIT DAN VIDEO DEMONSTRASI

6.1 Tautan Repositori Kode Sumber

Seluruh kode sumber, skrip otomasi pengujian, konfigurasi containerisasi Docker, arsitektur Protobuf, serta data mentah pengujian beban proyek mini ini dipublikasikan pada repositori Git resmi:

• **Tautan Repositori Resmi:** https://github.com/fathirrizkifadillah/RPC-CHAT-Tugas-KPT-.git

• **Cabang Utama (Branch):** main (versi stabil terverifikasi)

• **Dokumentasi Repositori:** Tersedia berkas README.md lengkap dengan instruksi instalasi lokal virtualenv, pengoperasian multi-container Docker Compose, serta eksekusi benchmark mandiri.

6.2 Rencana dan Skenario Video Demonstrasi

Sesuai ketentuan pengumpulan pada modul praktikum halaman 13, kelompok kami menyusun rencana demonstrasi sistem dengan durasi terencana maksimal 5 menit yang memperlihatkan fungsionalitas aplikasi dan pembuktian toleransi kegagalan:

• **Tautan Video Demonstrasi:** https://youtu.be/[ID_VIDEO_YOUTUBE] (Tautan siap diisi setelah rekaman demonstrasi diunggah)

• **Alokasi Batas Waktu:** Maksimal 5 Menit (Sesuai ketentuan batas waktu modul)

Tabel 6.1 Skenario Struktur Video Demonstrasi 5 Menit

Rentang Waktu (Timeline)	Segmen Demonstrasi	Aktivitas dan Pembuktian Fungsionalitas
00:00 – 00:45	Pendahuluan & Arsitektur	Perkenalan anggota kelompok dan penjelasan ringkas diagram arsitektur gRPC serta pembagian tugas.
00:45 – 02:00	Komunikasi Obrolan Real-Time	Menjalankan gRPC server (:50051), menghubungkan Client Alice dan Bob, serta mendemonstrasikan bidirectional streaming.
02:00 – 02:45	Fitur Unary RPC &	Menjalankan perintah /users untuk melihat

Rentang Waktu (Timeline)	Segmen Demonstrasi	Aktivitas dan Pembuktian Fungsionalitas
	ListUsers	registry online dan /unary untuk pengujian jalur pesan request-response.
02:45 – 03:30	Toleransi Kegagalan & Disconnect	Memutus koneksi salah satu client secara mendadak; membuktikan pembersihan otomatis server dan notifikasi leave ke client lain.
03:30 – 04:30	Pengujian Kinerja & Pytest	Mengeksekusi pytest (22 passed) dan skrip run_benchmark.py untuk mengukur latensi dan throughput 1, 5, dan 20 client.
04:30 – 05:00	Penutup & Kesimpulan	Pemaparan ringkas ketercapaian seluruh rubrik penilaian dan penutupan demonstrasi.

DAFTAR PUSTAKA

[1] Google LLC, "gRPC Python Documentation and Tutorials," 2024. [Online]. Tersedia: https://grpc.io/docs/languages/python/.

[2] Protocol Buffers Documentation, "Language Guide (proto3)," 2024. [Online]. Tersedia: https://protobuf.dev/programming-guides/proto3/.

[3] A. S. Tanenbaum dan M. Van Steen, Distributed Systems: Principles and Paradigms, 3rd ed. CreateSpace Independent Publishing Platform, 2017.

[4] M. van Steen dan A. S. Tanenbaum, Distributed Systems, 4th ed., 2023. [Online]. Tersedia: https://www.distributed-systems.net/.

[5] Tim Dosen Komputasi Paralel dan Terdistribusi, "Modul Latihan & Praktikum Remote Procedure Calls (RPC)," Jurusan Informatika, Fakultas Teknik, Universitas Siliwangi, 2026.

[6] M. Nygard, Release It!: Design and Deploy Production-Ready Software, 2nd ed. Pragmatic Bookshelf, 2018.

[7] Python Software Foundation, "concurrent.futures — Launching parallel tasks," Python 3.11 Documentation, 2024. [Online]. Tersedia: https://docs.python.org/3/library/concurrent.futures.html.

[8] IETF, "Hypertext Transfer Protocol Version 2 (HTTP/2)," RFC 7540, 2015. [Online]. Tersedia: https://datatracker.ietf.org/doc/html/rfc7540.

LAMPIRAN-LAMPIRAN

Lampiran A: Cuplikan Implementasi Kode Sumber Service Logic (src/chat_room/service.py)

Berikut adalah cuplikan implementasi utama LayananChatServicer yang menangani logika RPC Join, SendMessage, dan Chat:

```
class LayananChatServicer(pb_grpc.LayananChatServicer):
    def __init__(self, state: ChatState):
        self.state = state

    def Join(self, request: pb.JoinRequest, context:
grpc.ServicerContext) -> pb.JoinResponse:
        username = request.username.strip()
        if not username:
            context.abort(grpc.StatusCode.INVALID_ARGUMENT, "Username
tidak boleh kosong")

        user_id, err = self.state.add_user(username, request.client_id,
request.request_id)
        if err == "ALREADY_EXISTS":
            context.abort(grpc.StatusCode.ALREADY_EXISTS, f"Username
'{username}' sudah digunakan")

        return pb.JoinResponse(
            success=True,
            message=f"Selamat datang, {username}!",
            user_id=user_id,
            joined_at=int(time.time() * 1000)
        )

    def SendMessage(self, request: pb.SendMessageRequest, context:
grpc.ServicerContext) -> pb.SendMessageResponse:
        # Semantik At-Most-Once via deduplikasi PROSES
        cached = self.state.get_processed(request.request_id)
        if cached:
            return cached

        frame = pb.ChatFrame(
            type=pb.FRAME_TYPE_MESSAGE,
            request_id=request.request_id,
            sender=request.sender_username,
            content=request.content,
            timestamp=request.timestamp
        )
        self.state.broadcast_frame(frame)
        resp = pb.SendMessageResponse(
            request_id=request.request_id,
            success=True,
```

```
            message="Pesan berhasil dikirim",
            delivered_at=int(time.time() * 1000)
        )
        self.state.record_processed(request.request_id, resp)
        return resp
```

Lampiran B: Cuplikan Implementasi Kode Sumber State Management (src/chat_room/state.py)

Berikut adalah cuplikan implementasi ChatState yang menangani antrean terisolasi dan sinkronisasi RLock:

```
class ChatState:
    def __init__(self, queue_maxsize: int = 100, max_proses: int =
10000):
        self._lock = threading.RLock()
        self._users: Dict[str, Dict[str, Any]] = {}
        self._usernames: Dict[str, str] = {}
        self._queues: Dict[str, queue.Queue] = {}
        self._proses: OrderedDict[str, Any] = OrderedDict()
        self._queue_maxsize = queue_maxsize
        self._max_proses = max_proses

    def broadcast_frame(self, frame: pb.ChatFrame, exclude_user_id:
Optional[str] = None) -> int:
        with self._lock:
            target_queues = [(uid, q) for uid, q in self._queues.items()
if uid != exclude_user_id]

        delivered = 0
        for uid, q in target_queues:
            try:
                q.put_nowait(frame)
                delivered += 1
            except queue.Full:
                try:
                    _ = q.get_nowait()
                    q.put_nowait(frame)
                    delivered += 1
                except Exception:
                    pass
        return delivered
```

Lampiran C: Data Mentah Hasil Uji Beban Benchmark (JSON & CSV)

Berikut adalah rekaman data mentah aktual yang dihasilkan langsung oleh skrip benchmarks/run_benchmark.py:


```
{
  "timestamp": "2026-11-01 16:19:27",
  "server_address": "127.0.0.1:50051",
  "results": [
    {
      "num_clients": 1,
      "messages_per_client": 50,
      "total_requests": 50,
      "connected_clients": 1,
      "success_count": 50,
      "failure_count": 0,
      "total_duration_sec": 0.0734,
      "throughput_rps": 681.28,
      "avg_latency_ms": 1.098,
      "median_latency_ms": 0.941,
      "p95_latency_ms": 1.818,
      "min_latency_ms": 0.65,
      "max_latency_ms": 1.903
    },
    {
      "num_clients": 5,
      "messages_per_client": 50,
      "total_requests": 250,
      "connected_clients": 5,
      "success_count": 250,
      "failure_count": 0,
      "total_duration_sec": 0.2748,
      "throughput_rps": 909.86,
      "avg_latency_ms": 5.103,
      "median_latency_ms": 5.042,
      "p95_latency_ms": 7.071,
      "min_latency_ms": 1.534,
      "max_latency_ms": 8.562
    },
    {
      "num_clients": 20,
      "messages_per_client": 50,
      "total_requests": 1000,
      "connected_clients": 20,
      "success_count": 1000,
      "failure_count": 0,
      "total_duration_sec": 1.0117,
      "throughput_rps": 988.43,
      "avg_latency_ms": 19.075,
      "median_latency_ms": 19.088,
      "p95_latency_ms": 25.704,
```

```
      "min_latency_ms": 2.724,
      "max_latency_ms": 30.976
    }
  ]
}
```



```
num_clients,messages_per_client,total_requests,connected_clients,success_cou
nt,failure_count,total_duration_sec,throughput_rps,avg_latency_ms,median_lat
ency_ms,p95_latency_ms,min_latency_ms,max_latency_ms
1,50,50,1,50,0,0.0734,681.28,1.098,0.941,1.818,0.65,1.903
5,50,250,5,250,0,0.2748,909.86,5.103,5.042,7.071,1.534,8.562
20,50,1000,20,1000,0,1.0117,988.43,19.075,19.088,25.704,2.724,30.976
```